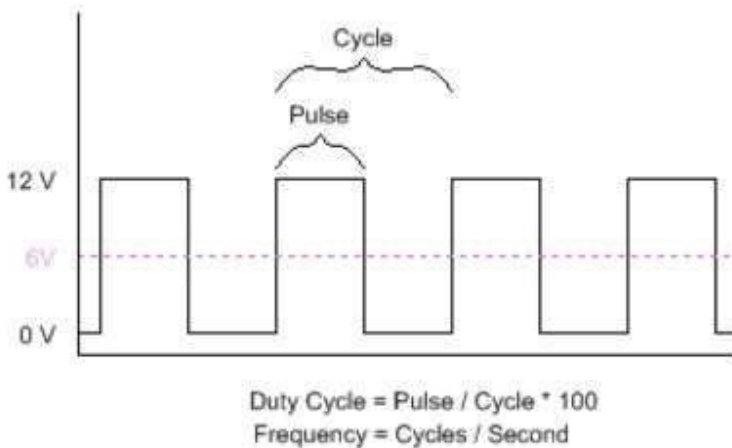


Короткі теоретичні відомості – необхідно для розуміння написаних далі розділів!



Тривалість активної частини імпульсу, тривалість активного стану імпульсу, тривалість активної частини сигналу, тривалість активного стану сигналу, duty cycle time – це все поняття, що означають час (у будь-яких одиницях) на протязі якого прямокутний сигнал знаходиться в стані HIGH (логічна одиниця). Якщо t_1 – момент часу зростаючого фронту, а t_2 – спадаючого, то duty cycle time розраховується як $t_2 - t_1$.

Тривалість період, період імпульсу, період сигналу – це все поняття, що означають час (у будь-яких одиницях), що проходить перед тим, як сигнал почне сам себе повторювати. Якщо t_1 – момент часу одного зростаючого фронту, а t_3 – момент часу наступного після нього, то період розраховується як $t_3 - t_1$.

Хід роботи

У лабораторній роботі № 5 досліджено три режими роботи периферії PIC18F4550: лічильник зовнішніх імпульсів на TMR1, режим захвату CCP1 та режим формування PWM (CCP2). За основу взято структуру проєкту з перериваннями, а далі послідовно реалізовано годинник реального часу, вимірювання тривалості імпульсу та керування шириною активного імпульсу із входу АЦП.

Я виконував дану роботу у її доволі ускладненому варіанті задля дослідження можливостей своїх, асемблера та Proteus. Тому в ній реалізовані деякі цікаві речі, про які далі буде сказано.

Далі будуть розглянуті найцікавіші ділянки коду саме для даного проєкту – те, чого не було у минулих лабораторних. Весь код займає більше 1100 рядків, тому його буде наведено у окремо надісланому файлі lab5.asm.

До протоколу також додано відеопояснення та демонстрацію, у якому може бути краще розкрито деякі моменти.

Перелік фрагментів, наведених та пояснених у даному протоколі:

- Організація коду та його структурний поділ на підпрограми
- Підпрограма початкового налаштування проєкту
- Ініціалізація таймерів та модулів CCP у потрібних режимах
- Підпрограми додаткових налаштувань модулів для кожного з прикладів
- Підпрограми обробки переривань TMR0 та TMR1 для прикладу 1
- Підпрограми обробки переривань TMR0 та TMR1 для прикладу 2 та 3
- Підпрограма обробки переривання CCP1 для прикладу 2 та 3
- Підпрограма обробки переривання АЦП та динамічного налаштування ШІМ сигналу
- Структурний алгоритм реалізації виводу 2-ох 6-розрядних чисел з врахуванням режиму заданого користувачем

1. Організація коду та його структурний поділ на підпрограми

Весь код розподілений по логічним структурним блокам, тому блоки з яких виконується керування виглядають приблизно так

```
83      start
84          call programSetup
85          call systemInit
86          call portsInit
87          call USARTInit
88          call TMR0Init
89          call TMR1Init
90      ;    call case2Init
91          call case3Init
92          goto main
93
```

Як можна бачити код стає читабельним навіть без коментарів.

Було використано такі стилі написання:

Для назв підпрограм – camelCase

Для назв змінних – snake_case

Інструкції – повністю малими літерами

Регістри та їх біти – повністю великими літерами

Також є умовний поділ на підпрограми ініціалізації _Init та обробки переривань _Interrupt.

2. Підпрограма початкового налаштування проекту

```
71      #define is_clock 0
72      #define output_device 1 ;set - output to terminal, cleared - output to 7seg
```

programSetup:

```
122     programSetup
123         bcf program_setup,is_clock
124         bsf program_setup,output_device
125
126         return
```

Це підпрограма, що може бути налаштована користувачем до початку компоновки проекту. Реалізована вона за рахунок розгалуджень і підпрограми виводу інформації на основі бітів `is_clock` та `output_device` у змінній `program_setup`.

3. Ініціалізація таймерів та модулів ССР у потрібних режимах

TMR0Init:

```
166      TMR0Init
167          movlw  b'10000111' ; Fosc/4; 16 bit mode; PS = 1:256
168          movwf  T0CON
169
170          bsf    INTCON , TMR0IE
171          bsf    INTCON2, TMR0IP
172
173
174          movlw  0xF0
175          movwf  TMR0H
176          clrf   TMR0L
177
178          return
```

TMR0 налаштовано однаково незалежно від прикладу. Він потрібен для виводу інформації та виклику перетворення АЦП з певним інтервалом.

Його налаштовано у 16-бітний режим, з використанням внутрішнього генератора частоти МК та подільника 1:256.

Дозволено переривання по переповненню та надання їм високого пріоритету.

Налаштовано початкове значення лічильника таймера.

TMR1Init:

```
181      TMR1Init
182          movlw b'10000111'
183          movwf T1CON
184
185      ;    call case1TMR1Init
186      ;    call case2TMR1Init
187      call case3TMR1Init
188
189      return
```

Для TMR1 використовується 16-бітний режим, тактування від зовнішнього джерела без синхронізації з тактами внутрішнього джерела.

Інші налаштування залежать від прикладу, в якому використовується таймер (їх наведено далі).

CCP1Init:

```
192      CCP1Init
193          bsf TRISC,RC2
194          movlw b'00000101'
195          movwf CCP1CON
196          bcf T3CON,T3CCP2
197
198          bsf PIE1,CCP1IE
199          bsf IPR1,CCP1IP
200
201          clrf ccp1_counter
202
203      return
```

CCP1 налаштовано в режим збору даних бо зростаючому фронту. TMR3 відключено від CCP1.

Надано дозвіл та високий пріоритет переривань.

TMR2Init:

```
206      TMR2Init
207          movlw b'00000101'
208          movwf T2CON
209
210          return
```

TMR2 увімкнено, в ньому використовується попередній подільник 1:4. Всі інші налаштування – по замовчуванню.

CCP2Init:

```
213      CCP2Init
214          bcf TRISC,RC1
215          movlw b'00001100'
216          movwf CCP2CON
217
218          return
```

Модуль CCP2 налаштовано в режим ШІМ.

Підпрограма налаштування характеристик ШІМ

```
221      PWMInit
222          decf pwm_period,w
223          movwf PR2
224
225          movff pwm_duty_cycle,CCPR2L
226
227          return
```

Задано початкові значення періоду та тривалості імпульсу ШІМ сигналу.

ADCInit:

```
231      ADCInit
232          bsf    PIE1, ADIE
233          bsf    IPR1, ADIP
234
235          movlw  b'00000001'
236          movwf  ADCON0
237
238          movlw  b'00001110'
239          movwf  ADCON1
240
241          movlw  b'00010010'
242          movwf  ADCON2
243
244          bsf    TRISA, AN0
245
246          return
```

АЦП дозволені переривання та надано їм високий пріорітет.

В модуль обрано канал AN0 (ADCON0) та налаштовано його як аналоговий вхід (ADCON1). Встановлено потрібний час вибірки та частоту дискретизації (ADCON2).

4. Підпрограми додаткових налаштувань модулів для кожного з прикладів

TMR0 в усіх прикладах використовується однаково, тому починаємо з TMR1.

case1TMR1Init:

```
415      case1TMR1Init
416          bsf    PIE1, TMR1IE
417          bsf    IPR1, TMR1IP
418
419          movlw  0x80
420          movwf  TMR1H
421          clrf   TMR1L
422
423          return
```

Для першого прикладу TMR1 встановлено у початкове значення 0x80.

Також йому дозволені переривання і вони високого пріорітету (використовується як лічильник секунд для годинника реального часу).

case2TMR1Init:

```
514 case2TMR1Init
515     clrf TMR1H
516     clrf TMR1L
517     bcf PIE1, TMR1IE
518     bcf IPR1, TMR1IP
519
520     return
```

В другому прикладі TMR1 рахує з 0.

До того ж йому заборонені переривання (тепер він використовується модулем CCP1 для збору даних – тривалості певних проміжків у тактах).

case3TMR1Init:

```
616 case3TMR1Init
617     call case2TMR1Init
618     movlw b'10000001'
619     movwf T1CON
620
621     return
```

Для 3го прикладу TMR1 вже тактується не від зовнішнього джерела, а від внутрішнього (щоб точніше рахувати тривалості саме в мкс, бо частота кристалу – 32 МГц, відповідно 1 мкс проходить за рівно 8 тактів TMR1).

Збережено налаштування із прикладу 2.

Далі наведено підпрограми додаткових налаштувань для прикладу 2 та 3:

case2Init:

```
498 case2Init
499     movlw b'01111010'
500     movwf multiplierH
501     movlw b'00010010'
502     movwf multiplierL
503
504     call CPP1Init
505
506     return
```

Для 2го прикладу множник для переведення з тривалості у тактах TMR1 в тривалість у мкс рівний 29450 (тактування від зовнішнього джерела).

Також викликається ініціалізація CCP1.

case3Init:

```
589 case3Init
590     call case2Init
591     call TMR2Init
592     call CCP2Init
593
594     movlw d'100'
595     movwf pwm_period
596     movlw d'25'
597     movwf pwm_duty_cycle
598     call PWMInit
599
600     call ADCInit
601
602     movlw b'00000000'
603     movwf multiplierH
604     movlw b'10000000'
605     movwf multiplierL
606
607     return
```

Для прикладу 3 додатково налаштовуються та встановлюються характеристики ШІМ сигналу.

Викликається ініціалізація АЦП.

Множник для переведення з тривалості у тактах TMR1 в тривалість у мкс рівний 128 (тактування від внутрішнього джерела).

5. Підпрограми обробки переривань TMR0 та TMR1 для прикладу 1

Нагадування – в прикладі 1 реалізовано годинник реального часу.

До того ж почну із загальної програм обробки переривань TMR0 та TMR1 (для всіх прикладів).

TMR0Interrupt та TMR1Interrupt:

<pre>248 TMR0Interrupt 249 movlw 0xF0 250 movwf TMR0H 251 clrf TMR0L 252 253 ; call case1TMR0Interrupt 254 ; call case2TMR0Interrupt 255 call case3TMR0Interrupt 256 257 call print 258 259 bcf INTCON, TMR0IF 260 261 return</pre>	<pre>264 TMR1Interrupt 265 ; call case1TMR1Interrupt 266 ; call case2TMR1Interrupt 267 call case3TMR1Interrupt 268 269 return</pre>
---	--

Бачимо, що після переривання в TMR0 завантажується початкове значення 0xF0 – таким чином реалізується певно швидкість оновлення виведеної інформації. Ця швидкість була скоригована експериментально.

Після цього викликається програма обробки переривань для певного прикладу (інші мають бути закоментовані). Ці програми форматують вхідні значення та записують їх у 24-ьох розрядні змінні output1 та output2. Вони є вхідними для процедури виводу print.

Бачимо, що для переривань по переповненню TMR1 немає спільних частин.

case1TMR0Interrupt:

```
426 case1TMR0Interrupt
427     clrf temp24_2
428     clrf temp24_1
429     clrf temp24_0
430
431     clrf mult16x16_arg1H
432     movff rtc_hours, mult16x16_arg1L
433     movlw b'00100111'
434     movwf mult16x16_arg2H
435     movlw b'00010000'
436     movwf mult16x16_arg2L
437     call mult16x16
438     movff mult16x16_res2,temp24_2
439     movff mult16x16_res1,temp24_1
440     movff mult16x16_res0,temp24_0
441
442     clrf mult16x16_arg1H
443     movff rtc_mins, mult16x16_arg1L
444     movlw b'00000000'
445     movwf mult16x16_arg2H
446     movlw b'01100100'
447     movwf mult16x16_arg2L
448     call mult16x16
449     movf mult16x16_res0,w
450     addwf temp24_0,f
451     movf mult16x16_res1,w
452     addwfc temp24_1,f
453     movf mult16x16_res2,w
454     addwfc temp24_2,f
455
456     movf rtc_secs,w
457     addwf temp24_0,f
458     movlw 0x00
459     addwfc temp24_1,f
460     movlw 0x00
461     addwfc temp24_2,f
462
463     movff temp24_0,output1_0
464     movff temp24_1,output1_1
465     movff temp24_2,output1_2
466     clrf output2_0
467     clrf output2_1
468     clrf output2_2
469
470     return
```

Цей блок коду форматує значення 3-ох змінних hours, minutes та seconds у 6-ти цифрове число за такою формулою:

$$\text{output1} = 10000 * \text{hours} + 100 * \text{minutes} + \text{seconds}.$$

В результаті отримаємо число формату hhmmss. Потрібно для сумісності із загальною програмою виводу інформації в обраний канал.

output2 встановлюється в 0.

case1TMR1Interrupt:

```
473 case1TMR1Interrupt
474     movlw 0x80
475     movwf TMR1H
476     clrf TMR1L
477     bcf PIR1,TMR1IF
478
479     incf rtc_secs,f
480     movlw .59
481     cpfsgt rtc_secs
482     return
483     clrf rtc_secs
484     incf rtc_mins,f
485     movlw .59
486     cpfsgt rtc_mins
487     return
488     clrf rtc_mins
489     incf rtc_hours,f
490     movlw .23
491     cpfsgt rtc_hours
492     return
493     clrf rtc_hours
494
495     return
```

Дана підпрограма при кожному переповненні таймеру знову встановлює в нього значення 0x80 та інкрементує змінну seconds, потім форматує. Якщо було переповнення цієї змінної, то інкрементуємо minutes, і так далі.

6. Підпрограми обробки переривань TMR0 та TMR1 для прикладу 2 та 3

Нагадування – приклад 2 це розрахунок тривалості та періоду імпульсу даного прямокутного сигналу. Приклад 3 – те ж саме + генерування власного ШІМ, тривалість імпульсу якого регулюється значенням на вході АЦП.

case2TMR0Interrupt:

```
523 case2TMR0Interrupt
524     movff count_delay1H, mult16x16_arg1H
525     movff count_delay1L, mult16x16_arg1L
526
527     movff multiplierH,mult16x16_arg2H
528     movff multiplierL,mult16x16_arg2L
529
530     call mult16x16
531
532     movff mult16x16_res2,time_delayH
533     movff mult16x16_res1,time_delayL
534 ;
535     bcf STATUS,C
536     rrcf time_delayH,f
537     rrcf time_delayL,f
538 ;
539     bcf STATUS,C
540     rrcf time_delayH,f
541     rrcf time_delayL,f
542
543     clrf output1_2
544     movff time_delayH,output1_1
545     movff time_delayL,output1_0
546
547
548     movff count_delay2H, mult16x16_arg1H
549     movff count_delay2L, mult16x16_arg1L
550
551     movff multiplierH,mult16x16_arg2H
552     movff multiplierL,mult16x16_arg2L
553
554     call mult16x16
555
556     movff mult16x16_res2,time_delayH
557     movff mult16x16_res1,time_delayL
558 ;
559     bcf STATUS,C
560     rrcf time_delayH,f
561     rrcf time_delayL,f
562 ;
563     bcf STATUS,C
564     rrcf time_delayH,f
565     rrcf time_delayL,f
566
567     clrf output2_2
568     movff time_delayH,output2_1
569     movff time_delayL,output2_0
570
571     return
```

Дана підпрограма форматує тривалість імпульсу та його період (збережені в count_delay1 та count_delay2) у тактах у тривалість в мкс за рахунок множення на потрібний множник multiplier.

case2TMR1Interrupt:

```
531 case2TMR1Interrupt
532 ;     clrf TMR1H
533 ;     clrf TMR1L
534     bcf PIR1,TMR1IF
535
536     return
```

Для прикладу 2 переривання TMR1 ніяк не використовувались.

7. Підпрограма обробки переривань CCP1 для прикладу 2 та 3

Ця підпрограма обробки переривань є спільною для 2-ох прикладів. Вона реалізує розрахунок тривалості імпульсу та його період за таким алгоритмом:

- 1) 2 рази зчитати зростаючий фронт, записуючи поточний час у тактах.

- 2) Знайшовши різницю 2-ох записаних моментів часу, знаходимо період імпульсу (від зростаючого до зростаючого фронту).
- 3) Перемикаємо на спадаючий фронт. При потраплянні на нього записуємо поточний час.
- 4) Знаходимо різницю з попереднім часом (2-ий фронт із пункту 1). Вийде тривалістю від зростаючого до спадаючого фронту – тривалість активного стану прямокутного сигналу.
- 5) Перевіраємо цю тривалість. Якщо вона вийшла більшою за період сигналу, то програма порахувала не найближчий спадаючий, а наступний після нього спадаючий фронт. Відповідно віднімаємо період, щоб знайти правильну тривалість активного стану сигналу.



CCP1Interrupt:

```
272     CCP1Interrupt
273         btfsc ccpl_counter,1
274         goto callCCP1Mode2
275         call CCP1InterruptMode1
276         return
277
278     callCCP1Mode2
279         call CCP1InterruptMode2
280
281         return
```

Підпрограма складається з 2-ох режимів.

1 – розрахунок тривалості від зростаючого фронту до зростаючого (для розрахунку періоду).

2 – розрахунок тривалості від зростаючого фронту до спадаючого (для розрахунку тривалості активного стану).

Вони перемикаються за рахунок лічильника.

Режим 1 – CCP1InterruptMode1:

<pre>284 CCP1InterruptMode1 285 movff CCPR1H,temp16H 286 movff CCPR1L,temp16L 287 288 movf ccpl_savedL,w 289 subwf temp16L,f 290 movf ccpl_savedH,w 291 subwfb temp16H,f 292 293 movff CCPR1H,ccpl_savedH 294 movff CCPR1L,ccpl_savedL 295 296 bcf PIR1,CCP1IF 297 incf ccpl_counter 298 299 btfss ccpl_counter,1 300 return</pre>	<pre>301 302 movff temp16H,count_delay2H 303 movff temp16L,count_delay2L 304 305 bcf PIE1,CCP1IE 306 btg CCP1CON,CCP1M0 307 bcf PIR1,CCP1IF 308 bsf PIE1,CCP1IE 309 310 return</pre>
--	--

Виконується 2 рази.

1-ий - виконається лише ліва частина, а саме збереження поточного на таймері часу.

2-ий - виконуються обидві частини, а саме збереження поточного часу і обрахунок тривалості у тактах (якщо пам'ятаєте, потім цю тривалість, форматує case2TMR0Interrupt).

Режим 2 – CCP1InterruptMode2:

```
313      CCP1InterruptMode2
314      movff CCP1H,temp16H
315      movff CCP1L,temp16L
316
317      clrf ccpl_counter
318
319      movf ccpl_savedL,w
320      subwf temp16L,f
321      movf ccpl_savedH,w
322      subwfb temp16H,f
323
324      movff temp16H,count_delay1H
325      movff temp16L,count_delay1L
326
327      movf count_delay2L,w
328      subwf temp16L,f
329      movf count_delay2H,w
330      subwfb temp16H,f
331
332      btfss STATUS, C
333      goto exitCCP1InterruptMode2
334
335      movff temp16H,count_delay1H
336      movff temp16L,count_delay1L
337
338      goto exitCCP1InterruptMode2
339
340      exitCCP1InterruptMode2
341      bcf PIE1,CCP1IE
342      btg CCP1CON,CCP1M0
343      bcf PIR1,CCP1IF
344      bsf PIE1,CCP1IE
345
346      return
```

Виконується одноразово, після чого знову починається цикл з 1-го режиму.

В лівій частині відбувається збереження поточного часу та обрахунок тривалості між спадаючим та зростаючим фронтом (віднімаємо від поточного часу попередній).

В кінці лівого скріншота перевіряємо чи не вийшла наша тривалість більшою за період.

Якщо так, то треба додатково відняти й період.

В обох випадках переходимо до exitCCP1InterruptMode2 що відповідає за перезапуск CCP1 в потрібному 1-му режимі.

8. Підпрограма обробки переривання АЦП та динамічного налаштування ШІМ сигналу

ADCInterrupt:

```
349  ADCInterrupt
350      movf pwm_period,w
351      mulwf ADRESH
352
353      movff PRODH,pwm_duty_cycle
354      decf pwm_period,w
355      cpfseq pwm_duty_cycle
356      incf pwm_duty_cycle
357      bcf CCP2CON,5
358      bcf CCP2CON,4
359      movff pwm_duty_cycle,CCPR2L
360
361      bcf PIR1,ADIF
362
363      return
```

Дана підпрограма викликається після опрацювання напруги на вході АЦП. Я використовую лише старший розряд ADRESH, відповідно від АЦП я отримую значення від 0 до 255. Після цього множенням на період я формую це значення. Це означає, що незалежно від заданого періоду ШІМ, його тривалість імпульсу буде регулюватись у тому ж масштабі, що й напруга на вході АЦП!

Якщо виходить так, що duty cycle рівний 0 або періоду обрізаю їх до значень 1 та (період – 1) відповідно. Таким чином я уникаю нештатних ситуацій.

9. Структурний алгоритм реалізації виводу 2-ох 6-розрядних чисел з врахуванням режиму заданого користувачем

Підпрограма print (відповідає за вивід інформації) разом із своїми службовими підпрограмами займає більше 450 рядків коду, тому не буду повністю її тут наводити, до того ж вона в основному складається з повторюваних блоків коду, частина з яких будуть розглянуті.

print:

```
619  print
620      movff output1_2,bin_value_24_2
621      movff output1_1,bin_value_24_1
622      movff output1_0,bin_value_24_0
623
624      call binToBCD
625
626      movff bcd_value_32_2,output1_2
627      movff bcd_value_32_1,output1_1
628      movff bcd_value_32_0,output1_0
629
630      movff output2_2,bin_value_24_2
631      movff output2_1,bin_value_24_1
632      movff output2_0,bin_value_24_0
633
634      call binToBCD
635
636      movff bcd_value_32_2,output2_2
637      movff bcd_value_32_1,output2_1
638      movff bcd_value_32_0,output2_0
639
640      call bcdToSegments
641
642      btfss program_setup,output_device
643      call print7seg
644
645      btfsc program_setup,output_device
646      call printUSART
647
648      return
```

Print виконує даний алгоритм:

- 1) Запис отриманих від блоків обробки (TMR0Interrupt) чисел у змінну bin_value (вона є 24-ьох розрядною).
- 2) Перетворення двійкових чисел (bin_value) в двійково-десяткові (bcd_value). Це робить підпрограма binToBCD, взята та модифікована (для роботи з 24-ьох розрядними числами) з методички.
- 3) Тепер наші 2 числа (наприклад, 2 тривалості) зберігаються у змінних output1 та output2 (вони є 24-ьох розрядними, тому розділені на 3 8-ми бітні комірки пам'яті). Вони в такому форматі:

outputX (де X = 1 або 2):

a a a a b b b b - c c c c d d d d - e e e e f f f f

Де a, b, c, d, e, f – тетради (їх 6), в кожній з яких зберігається цифра 6-ти цифрового числа.

Наприклад число 123456:

0 0 0 1 0 0 1 0 - 0 0 1 1 0 1 0 0 - 0 1 0 1 0 1 1 0
1 2 3 4 5 6

- 4) Записуємо кожну з тетрад в окремі змінні – змінні цифр вихідного числа, що буде виводитись на індикатор або у термінал. Це робить підпрограма bcdToSegments (початок код на скріншоті зправа).
- 5) В залежності від налаштувань користувача (пам'ятаєте, підпрограма programSetup) виводимо збережені 6 цифр або а термінал, або на індикатори (ми можемо вивести максимум 2 числа по 6 цифр).

bcdToSegments:

```
699      bcdToSegments
700      movlw b'11110000'
701      andwf output1_2,w
702      movwf temp
703      swapf temp,w
704      movwf output1_seg1
705
706      movlw b'00001111'
707      andwf output1_2,w
708      movwf output1_seg2
```

Підпрограма складається з 6 повторень блоку, що я навів для всіх тетрад 2-ох чисел. Вона має записати всі тетрад в кожну окрему змінну.

Працює запис тетрад таким чином:

- 1) Якщо тетрада в старших 4-ьох бітах – виділяємо її маскою 1 1 1 1 0 0 0 0 (виконуємо логічне множення).

Після цього отримаємо: a3 a2 a1 a0 0 0 0 0.

Нам треба отримати ці ж 4 розряди в молодших бітах. Інструкція swapf «свапає» місцями 4 старших та 4 молодших розряди комірки пам'яті.

В результаті отримаємо: 0 0 0 0 a3 a2 a1 a0.

Це значення й збережемо в змінну.

- 2) Якщо тетрада в молодших 4-ьох бітах – виділяємо її маскою 0 0 0 0 1 1 1 1.

Після цього отримаємо: 0 0 0 0 b3 b2 b1 b0.

Додаткових маніпуляцій виконувати не доводиться.

print7seg:

```
763     print7seg
764
765     setSeg1
766         call delaySmall
767         movlw b'00110000'
768         iorwf output1_seg1,w
769         clrf PORTD
770         movwf PORTD
771
772         btfsc program_setup,is_clock
773         goto setSeg2
774
775         movlw b'00110000'
776         iorwf output2_seg1,w
777         clrf PORTB
778         movwf PORTB
```

print7seg – безпосередньо виводить значення на індикатор з вибором потрібного розряду в старшій тетраді, та записом значень із змінних попереднього етапу в молодшу тетраду, далі це напряму відправляється в PORTD (порт, що підключений до індикатора, який відображає 1-ше число) або PORTB (порт, що підключений до індикатора, який відображає 2-ге число).

Наведений блок повторується 6 разів – для кожної цифри числа 1 та числа 2.

Зверніть увагу на розгалудження – перевірямо чи обраний користувачем режим годинника – тоді будемо виводити лише 1-ше число, а вивід 2-го – пропустимо.

printUSART:

```
871 | printUSART
872 |     call delaySmall
873 |
874 | checkTXClear
875 |     btfss TXSTA,TRMT
876 |     goto checkTXClear
877 |
878 |     movlw '\f'
879 |     movwf TXREG
880 |
881 | checkTX1
882 |     btfss TXSTA,TRMT
883 |     goto checkTX1
884 |
885 |     movlw 0x30
886 |     addwf output1_seg1,w
887 |     movwf TXREG
888 |
889 | checkTX2
890 |     btfss TXSTA,TRMT
891 |     goto checkTX2
892 |
893 |     movlw 0x30
894 |     addwf output1_seg2,w
895 |     movwf TXREG
896 |
897 |     btfss program_setup,is_clock
898 |     goto checkTX3
899 |
900 | checkTXColon1
901 |     btfss TXSTA,TRMT
902 |     goto checkTXColon1
903 |
904 |     movlw ':'
905 |     movwf TXREG
```

Підпрограма по черзі виводить цифри збережені у змінних після перетворення в bcdToSegments (для цього до них треба додати символ «0» або 0x30).

Зверніть на розгалуження на скріншоті праворуч – якщо користувачем обраний режим годинника, то виведемо « : » (двокрапка), інакше – пропустимо цю частину коду.

Окрім цього на початку виводу надсилаємо в USART «\f» - символ очищення терміналу.

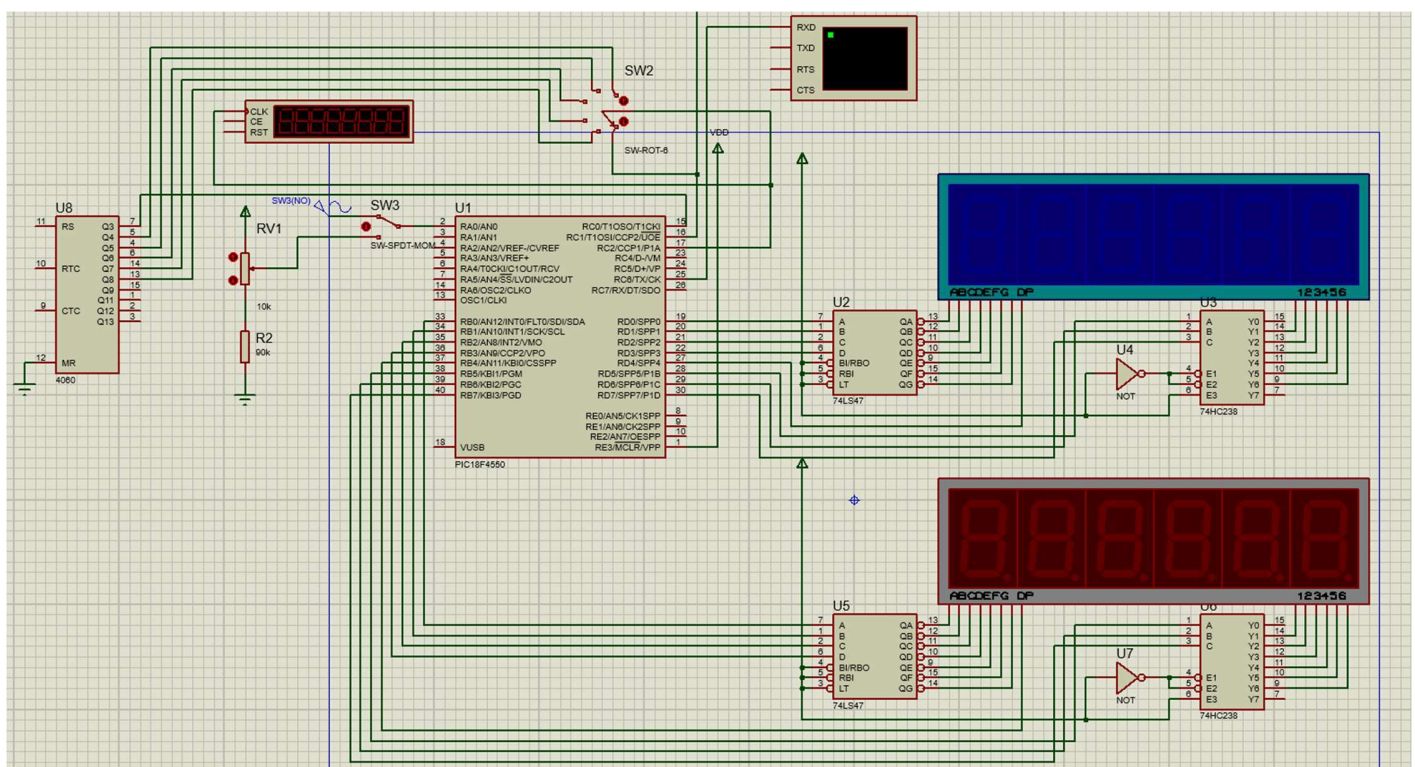
Після першого числа надсилаємо в USART «\r» - символ нового рядка у терміналі.

Схема підключення та результати моделювання

Далі будуть наведені скріншоти, що демонструють:

- Стандартну схему для виконання даної ЛР
- Формат виводу годинника реального часу у терміналі
- Формат виводу тривалості активної частини та періоду імпульсу (в мкс) у терміналі
- Результиуючий ШІМ сигнал, параметри якого відповідають відображеним тривалостям у терміналі
- Осцилограму вихідного сигналу після RC-ланки (ФНЧ), на вхід якої подано модульований синусоїдою ШІМ сигнал.
- Графіки, що забражають сигнали на вході та виході RC-ланки
- Активно використовувані інструменти Proteus для дебагу
- Ефективні рішення для зручного користування схемою під час симуляції

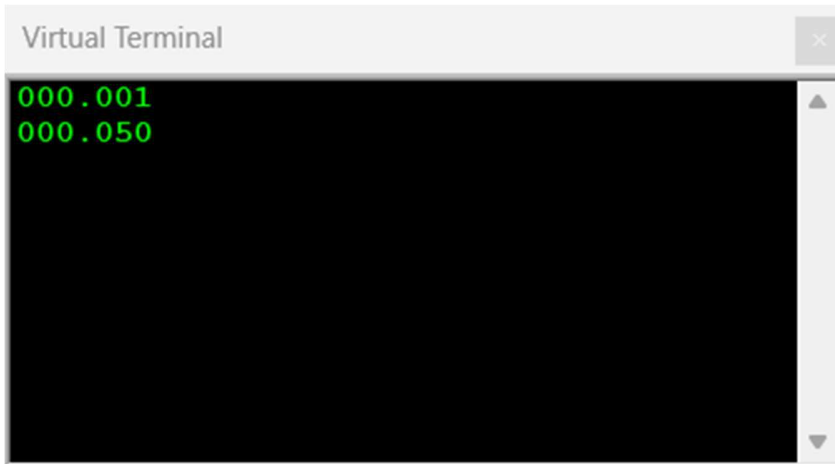
1. Схема лабораторної роботи



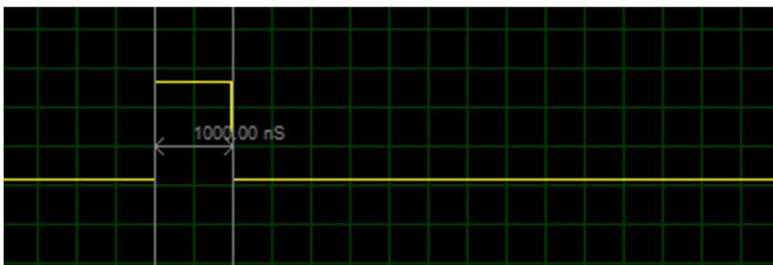
2. Годинник реального часу у терміналі



3. Вимірювання імпульсу та PWM – вивід у термінал



4. Осцилограма та порівняння PWM з розрахованими значеннями

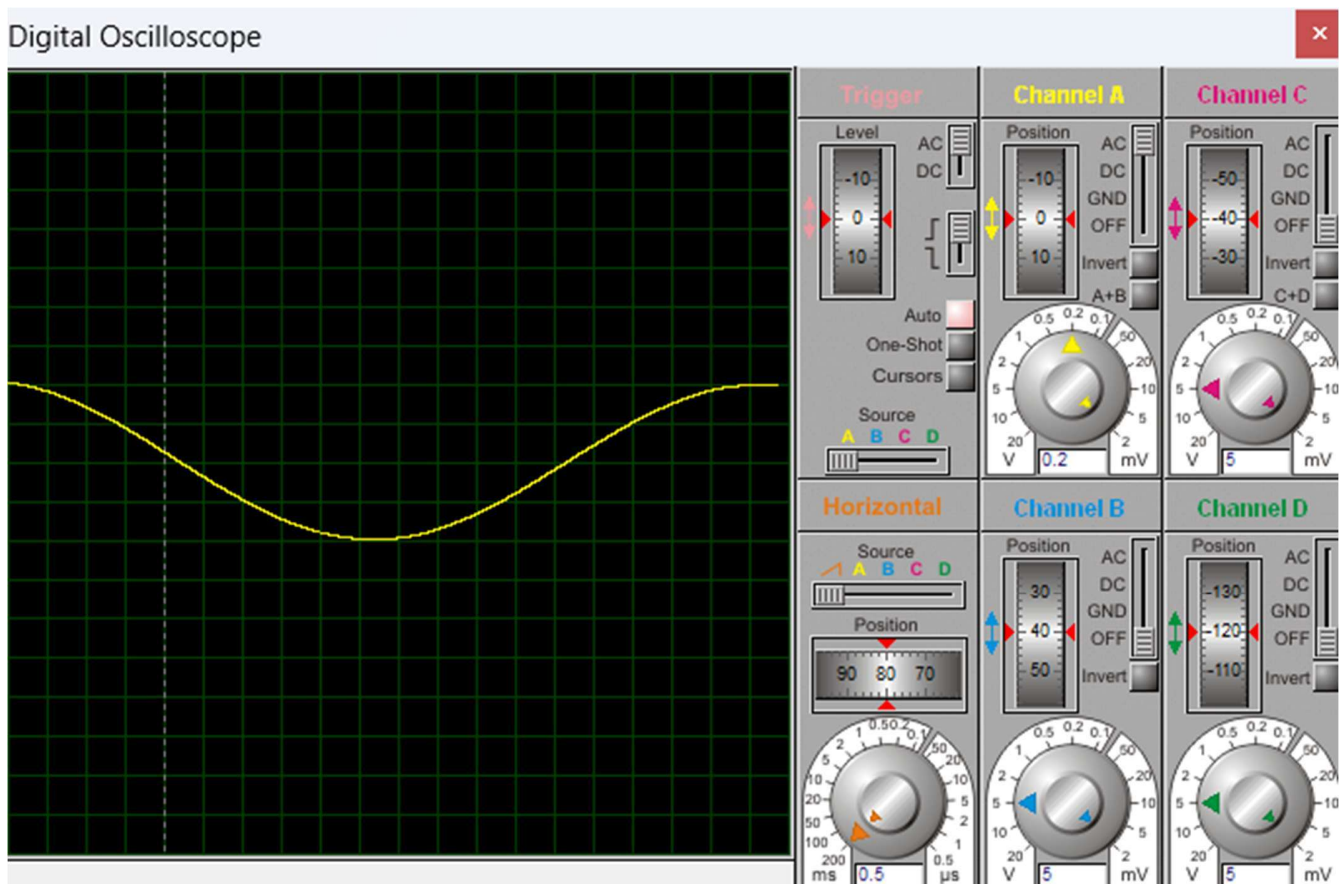


Тривалість активної частини імпульсу – 1 мкс.



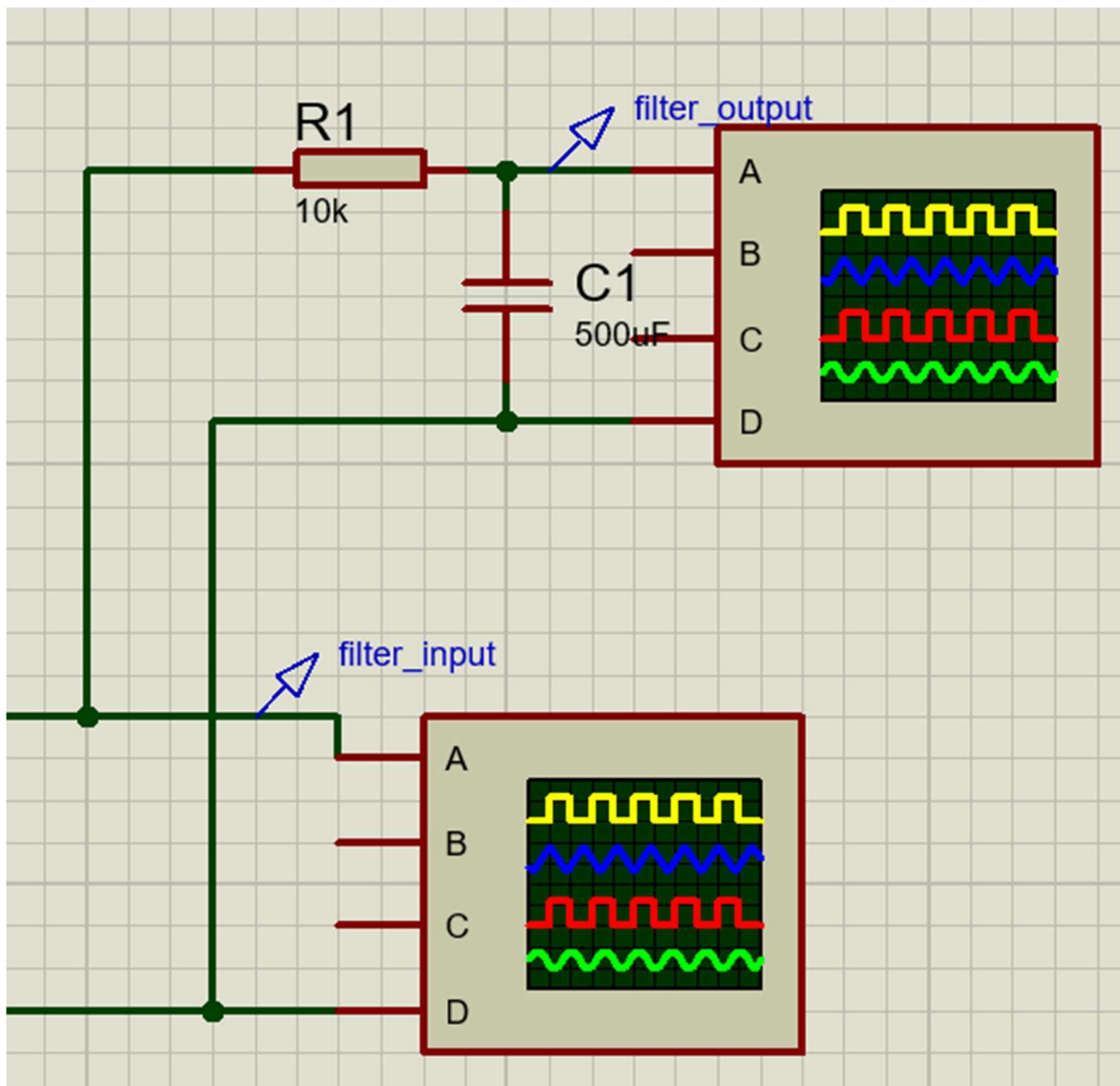
Період імпульсу – 50 мкс (частота – 20 кГц).

5. Осцилограма сигналу на виході ФНЧ, на вхід якого поданий модульований PWM



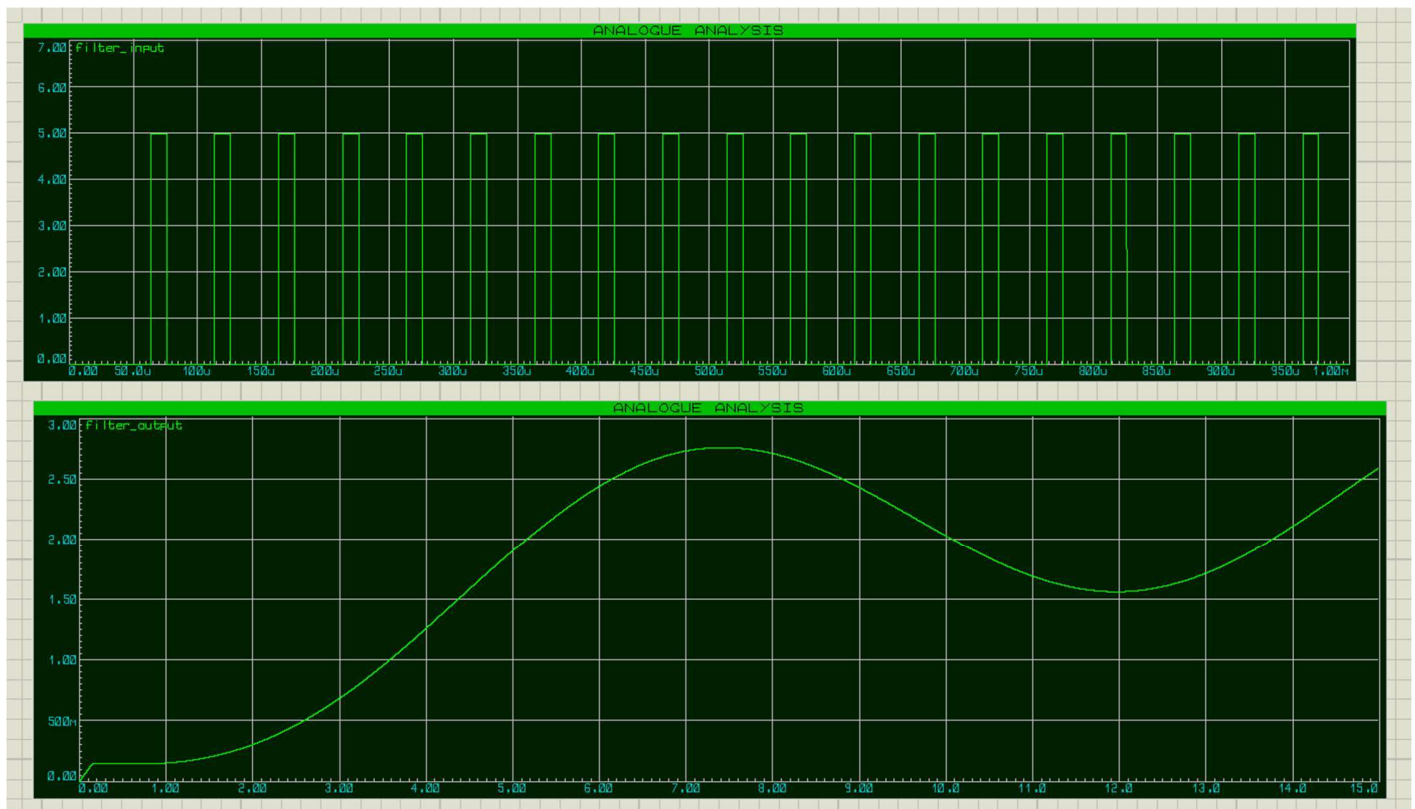
Бачимо красиву, зглажену синусоїду.

6. Графіки сигналів на вході та виході ФНЧ



`filter_input` – сигнал на вході ланки (верхній графік – ШІМ сигнал)

`filter_output` – на виході (нижній графік – результуюча синусоїда)



Сталу часу ланки, було підлаштовано таким чином, щоб отримати зглажену синусоїду.

7. Активно використані інструменти Proteus для дебагу

PIC18 CPU Source Code - U1

D:\Artem\IT\MPAsm_Projects\Lab5_Kr\

```

85      movwf output2_2
86      movlw b'10010000'
87      movwf output2_1
88      movlw b'00010010'
89      movwf output2_0
90
91      call programSetup
92      call systemInit
93      call portsInit
94      call USARTInit
95      call TMR0Init
96      call TMR1Init
97      ; call case2Init
98      call case3Init
99      goto main
100
101
102      main
103
104      goto main
105
106
107      highInt
108      btfsc INTCON,TMR0IF
109      call TMR0Interrupt
110
111      btfsc PIR1,ADIF
112      call ADCInterrupt
113
114      btfsc PIR1,TMR1IF
115      call TMR1Interrupt
116
117      btfsc PIR1,CCP1IF
118      call CCP1Interrupt
119
120      retfie
121
122
123      lowInt
          
```

PIC18 CPU Registers - U1

PC: \$00000048	CYCLES: 4607367	W.MSGS: 0
INSTR.: GOTO 0x000048		
WREG: 73 \$49 %01001001	STATUS: ----	INTS: HL
TBLPTR: \$000000 BSR: 0	TOS: \$000000	SP: 0
FSR0: \$003D FSR1: \$0000	FSR2: \$0000	
LAT A: 0 \$00 %00000000	TRIS A: 1 \$01 %00000001	
LAT B: 0 \$00 %00000000	TRIS B: 0 \$00 %00000000	
LAT C: 0 \$00 %00000000	TRIS C: 181 \$B5 %10110101	
LAT D: 0 \$00 %00000000	TRIS D: 0 \$00 %00000000	
LAT E: 0 \$00 %00000000	TRIS E: 7 \$07 %00000111	

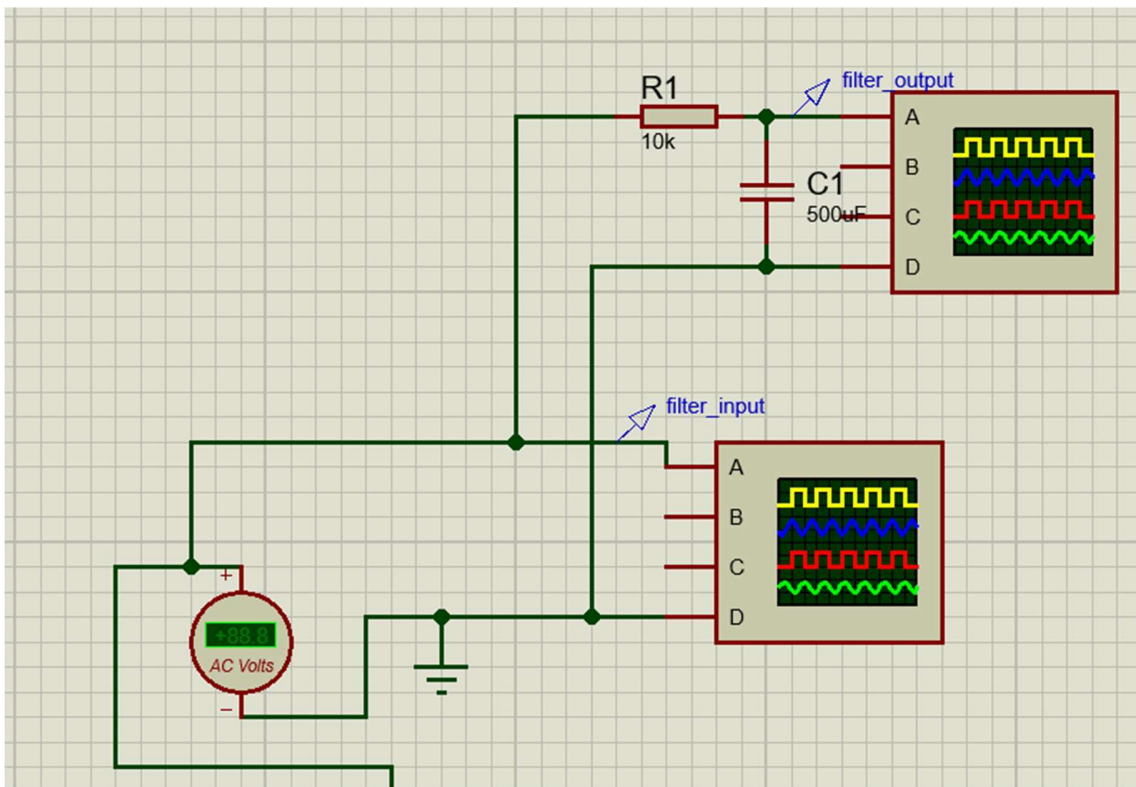
Watch Window			
Name	Value	Previous Value	Watch Expression
WREG	73	1	
ADRESH	0b00000111	0b00000100	on Change
ADRESL	0b00000000	0b11000000	on Change
Internal Timer1	19766	19726	= 0xffff
CCPR1 (WORD)	19714	35662	
CCPR1H	77	139	
CCPR1L	2	78	
PRODH	2	0	on Change
PRODL	188	128	on Change
TMR0H	240		= 0xff
TMR0L	77	76	= 0xff
TMR1H	0		= 0xff
TMR1L	54	14	= 0xff

На першому скріншоті – хід виконання коду, що можна відслідковувати в реальному часі, можна встановлювати умови зупину.

На другому – внутрішні регістри МК.

На третьому – вікно відслідковування потрібних регістрів та змінних, до того ж можна встановити умови зупину.

8. Ефективні рішення для зручного користування схемою під час виконання програми

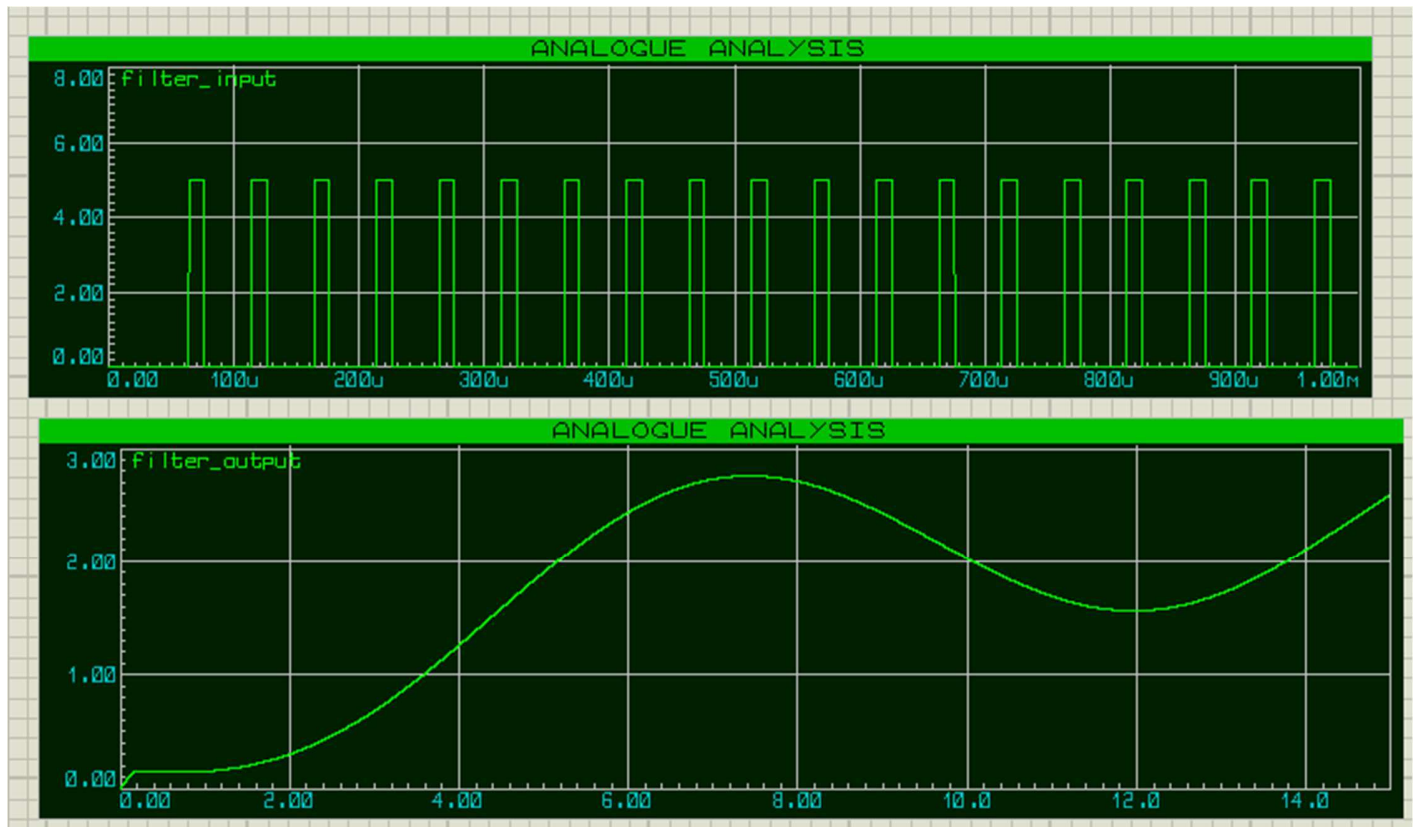


Вольтметр змінної напруги та 2 осцилоскопи використовувались для аналізу результуючого ШІМ сигналу.

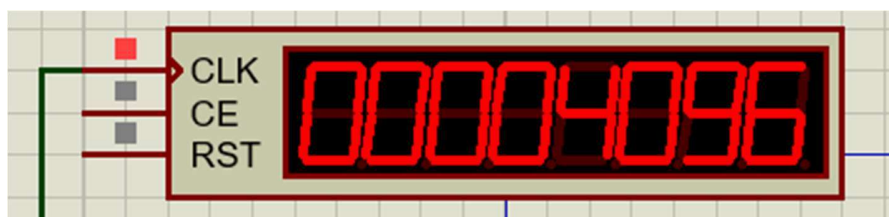
Вольтметр допомагає перевірити діюче значення напруги.

Перший осцилоскоп – перевірити тривалість активної частини імпульсу та його період.

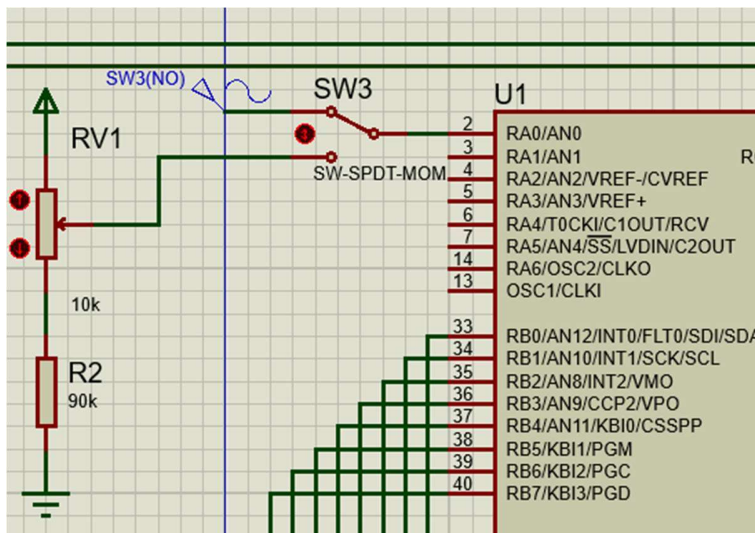
Другий осцилоскоп – перевірити відповідність змін тривалості активної частини ШІМ до встановлюваних рівнів напруги на АЦП.



Явне відображення сигналів з обох сторін ФНЧ.



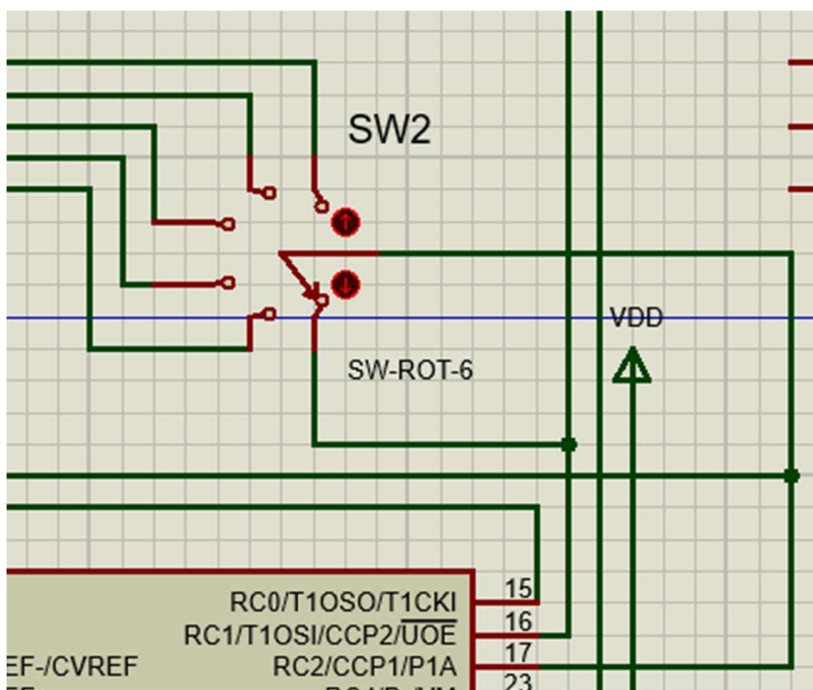
Аналізатор частот– перевірка відповідності частоти ШІМ та інших генераторів частот до заданих значень.



Перемикання вхідного сигналу до АЦП.

Один випадок – подільник напруги з регульованим потенціометром.

Другий – генератор змінної синусоїдальної напруги.



Це 6-ти позиційний перемикач, підключений до входу модуля збору інформації (CCP1). Використовується як канал який розраховує характеристики прямокутних сигналів – тривалість активної частини та періоду.

До цього перемикача підведені 5 лінії поділеної частоти від генератора частоти 32768 Гц.

Остання нижня лінія – сигнал ШІМ, згенерований моделем CCP2.